

DHA Suffa University
Department of Computer Science
Final Year Project

P80F25 STRATBOT
Software Design Specifications



Submitted by

Zaafir Ejaz Ur Rehman Siddiqui (CS221222)
Ebad Ahmed (CS221217)
Fatima Ather Rajput (CS221270)

Supervisor

Dr. Huma Jamshed

Document Information

Project Title	STRATBOT: AI-Based Formula 1 Race Strategy Simulation
Project Code	P80F25
Document Name	Software Design Specifications
Document Version	2.0 (Updated Implementation)
Document Identifier	P80F25-SDS
Document Status	Final
Author(s)	Zaafir Ejaz (Telemetry/ML), Ebad Ahmed (Data/ML), Fatima Ather Rajput (A
Approver(s)	Dr. Huma Jamshed
Issue Date	June 2026

Contents

1	Introduction	2
2	Overall System Description	2
2.1	Project Background & Scope	2
2.2	System Context / Architecture	2
3	External Interface Requirements	3
4	Functional Design	3
4.1	Use Cases (Updated)	3
5	Non-Functional Design	3
6	Detailed System Design	3
6.1	Data Design	3
6.2	UI / GUI Design (Implemented)	3
6.3	3D Car Customizer Detailed Design (FYP-II Addition)	8
6.4	RaceEngine Fast-Complete Design (Critical Fix)	8
7	References & Appendices	8
8	Appendices – Additional Captures & Diagrams	10

1 Introduction

This updated SDS (v2.0) documents the **as-built** design of the delivered StratBot system (June 2026), including all FYP-I core plus implemented FYP-II elements: the 3D interactive CarCustomizer, ML deltas and customStats driving the simulation, fast-complete with visible overlay, and rich PostRace analysis.

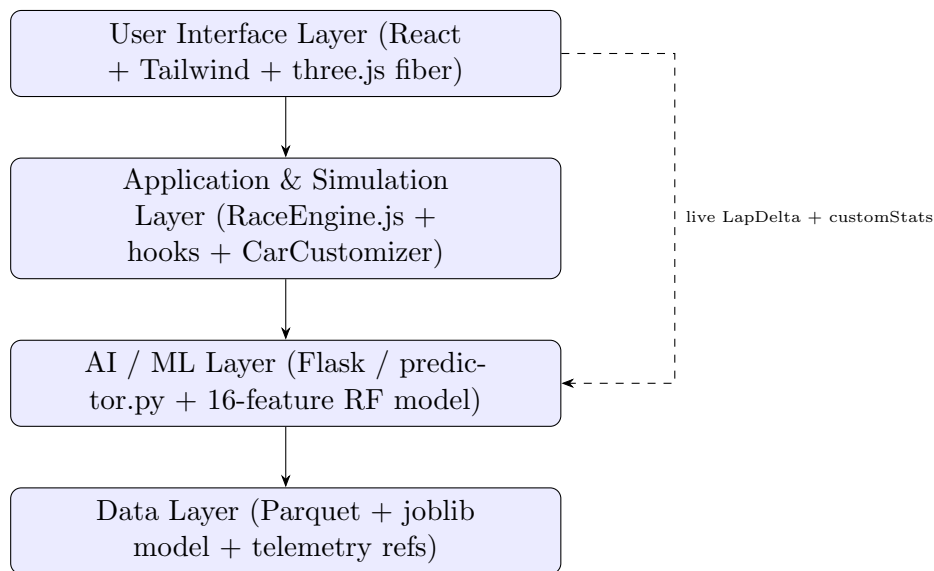
2 Overall System Description

2.1 Project Background & Scope

StratBot is a browser-based advisory platform for F1 strategy exploration. It combines a real-data ML pipeline (FastF1 2018-2025 + weather), a client-side high-fidelity simulation engine, live model inference, and an interactive 3D setup tool.

2.2 System Context / Architecture

The high-level layers (as implemented):



Key runtime components:

- `PreRaceSetup.jsx` + `CarCustomizer.jsx` (3D meshes + OrbitControls + live stats → visual + `onStatsChange`)
- `RaceEngine.js` (`useRaceEngine` hook, refs for `isFastCompleting`, `customCarStatsRef`, burst loop with `setTimeout` + `while(lapProgress)`, conditional state updates every 2 laps during fast)
- `useStratBotModel.js` + `Flask /api/predict/lap-delta`
- `App.jsx` (phase machine `BOOT` → `SETUP` → `RACING` → `POST_RACE`, overlay for `isFastCompleting`)
- `PostRaceSummary.jsx` (custom car stats box + ML table)

3 External Interface Requirements

(Identical to SRS; browser + localhost:5000/5173 + parquet on J: drive.)

4 Functional Design

4.1 Use Cases (Updated)

Primary actors: Analyst (Student/Fan), Admin (demo).

Key implemented flows:

1. Configure race + 3D custom car for tracked driver → Launch → (optional) interact with turns → Complete Full (visible overlay) → inspect PostRace (custom impact + ML).
2. Switch model variant / weather / compound at setup → observe different live predictions and post-race tables.
3. Fast sim auto-records “AUTO” decisions for any turns passed during burst.

(Old use-case diagram from original SDS is retained in images/ for reference; actual UI flows are captured in the screenshots below.)

5 Non-Functional Design

Performance target for fast complete met by 120-tick / 10ms burst + ref-only heavy work + every-2-lap state refresh.

6 Detailed System Design

6.1 Data Design

- Primary: `f1_model_ready_2018_2025.parquet` (16 cols including weather avgs/maxes)
- Model: `lap_delta_model.joblib` + `model_meta.json` (produced by `train_export.py`)
- Runtime: in-memory telemetry refs per driver, `customCarStats` attached only to tracked driver object.

No traditional RDBMS (scope); Parquet + joblib as specified for FYP-I.

6.2 UI / GUI Design (Implemented)

All screenshots below are live captures from the running June 2026 build (not mockups).

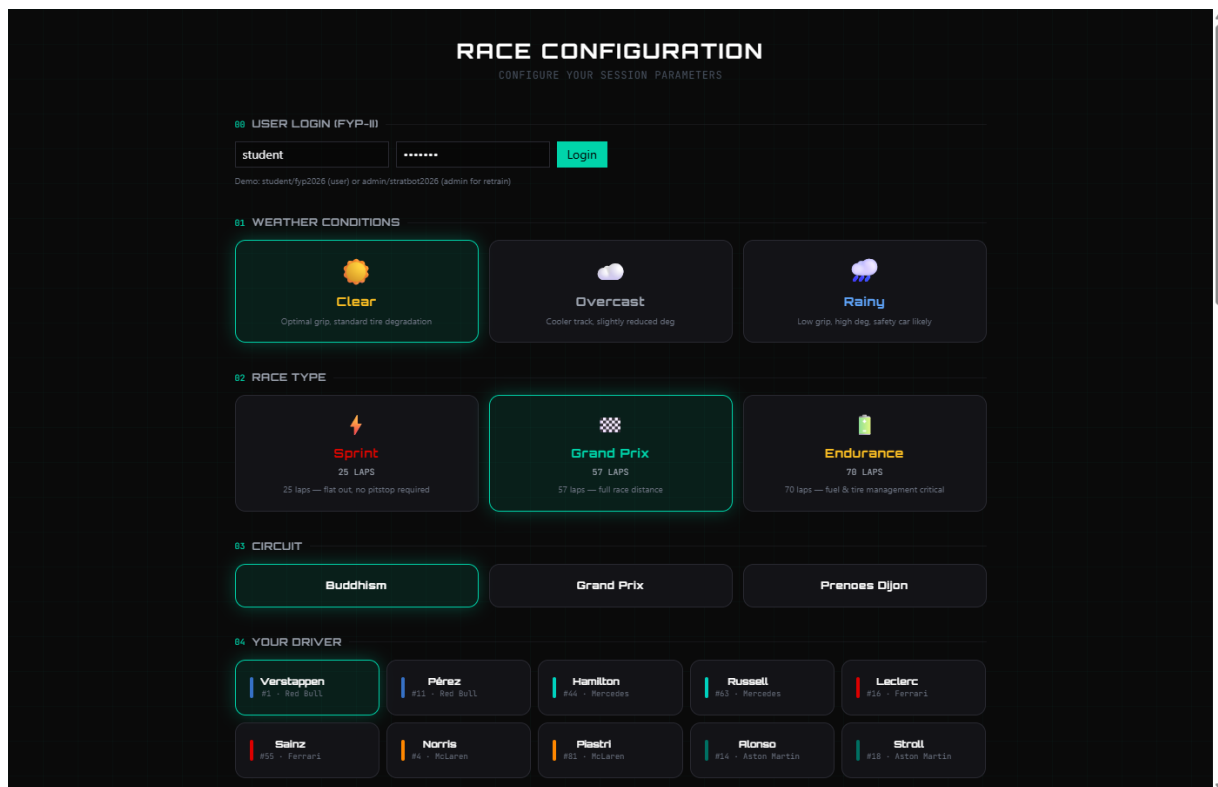


Figure 1: Pre-Race Setup (Section 5 in original SDS) — driver cards + fully functional embedded 3D F1 CarCustomizer. Clicking parts cycles compounds; sliders update live 3D (wing angle, tyre scale/color, engine glow) and the stats object passed to startRace().

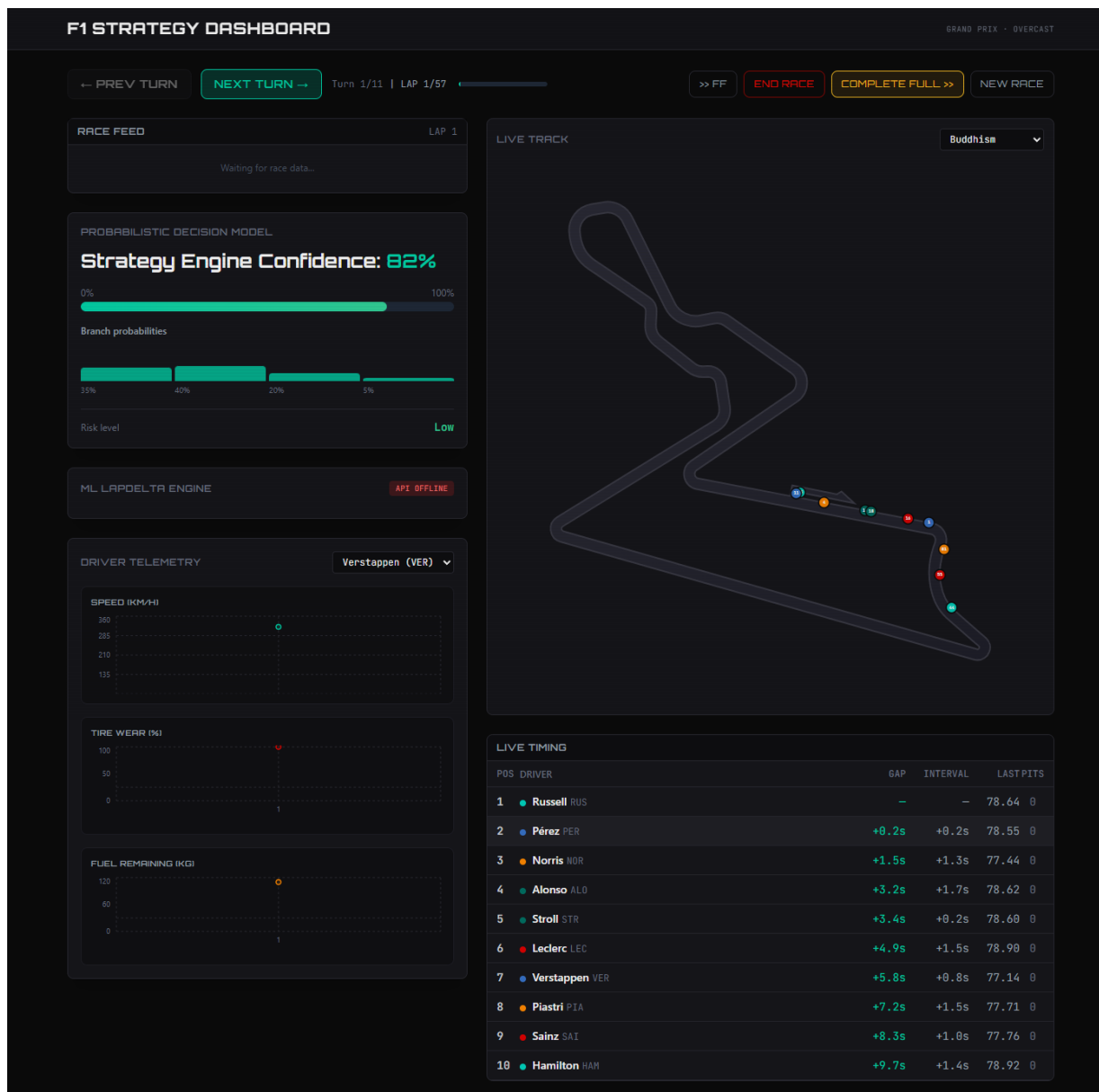


Figure 2: Racing screen with TurnNavigation (top progress bar, always-on prev/next, Complete Full, FF), LiveTimingLeaderboard, TrackMap, TelemetryCharts, ModelInsightsPanel (live prediction).

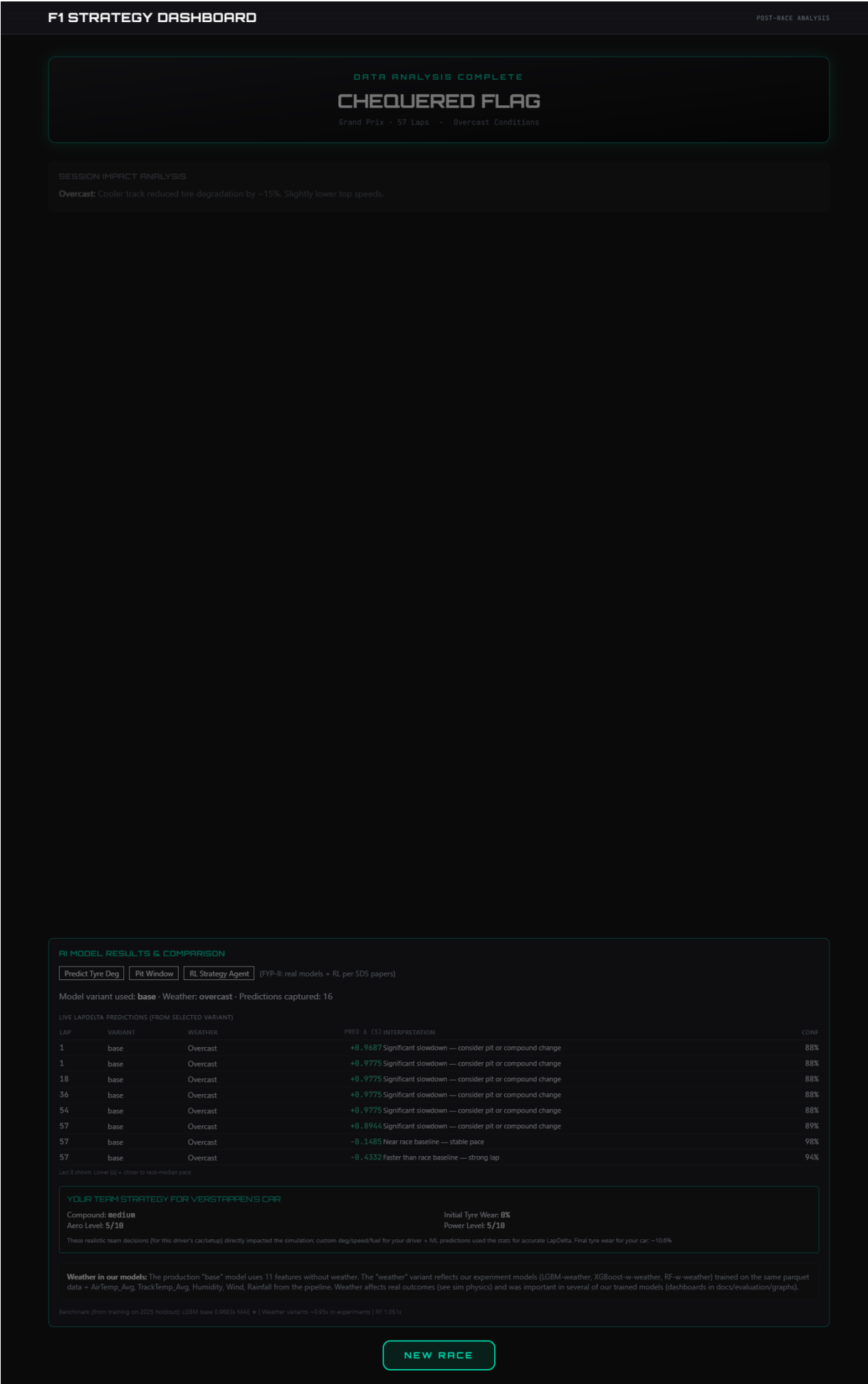
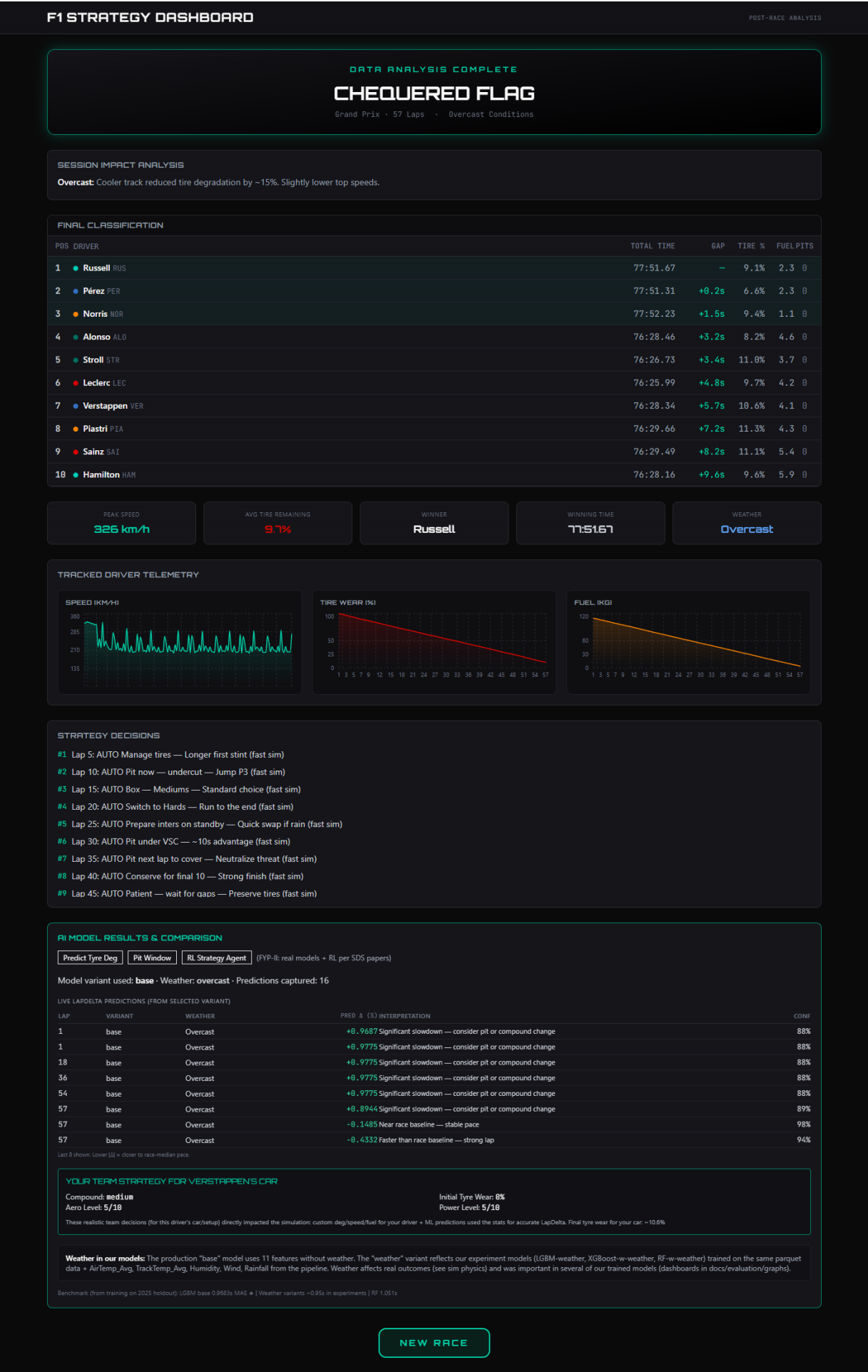


Figure 3: Fast-complete loading overlay (z-[60] fixed inset). Text, live lap, animated progress bar. The 16ms yield + periodic setDrivers during burst ensures it is always visible and updates.



6.3 3D Car Customizer Detailed Design (FYP-II Addition)

- Uses @react-three/fiber + drei (OrbitControls, Html).
- Procedural meshes: body (box + cylinders for sidepods), front/rear wings (angled planes), nose, tyres (cylinders whose scale/color change with wear/compound), engine cone with emissive glow for power.
- onPartClick cycles compound or opens simple numeric controls.
- applyTeamRecommendation() computes realistic defaults per weather/track (e.g. higher aero in wet, power for overtaking tracks).
- Live sync: stats → mesh material/scale/rotation + parent onStatsChange → Pre-RaceSetup state → config.carStats → RaceEngine buildInitialDrivers + predictor .build_row (adds biases to TyreLife, aero, power columns).

6.4 RaceEngine Fast-Complete Design (Critical Fix)

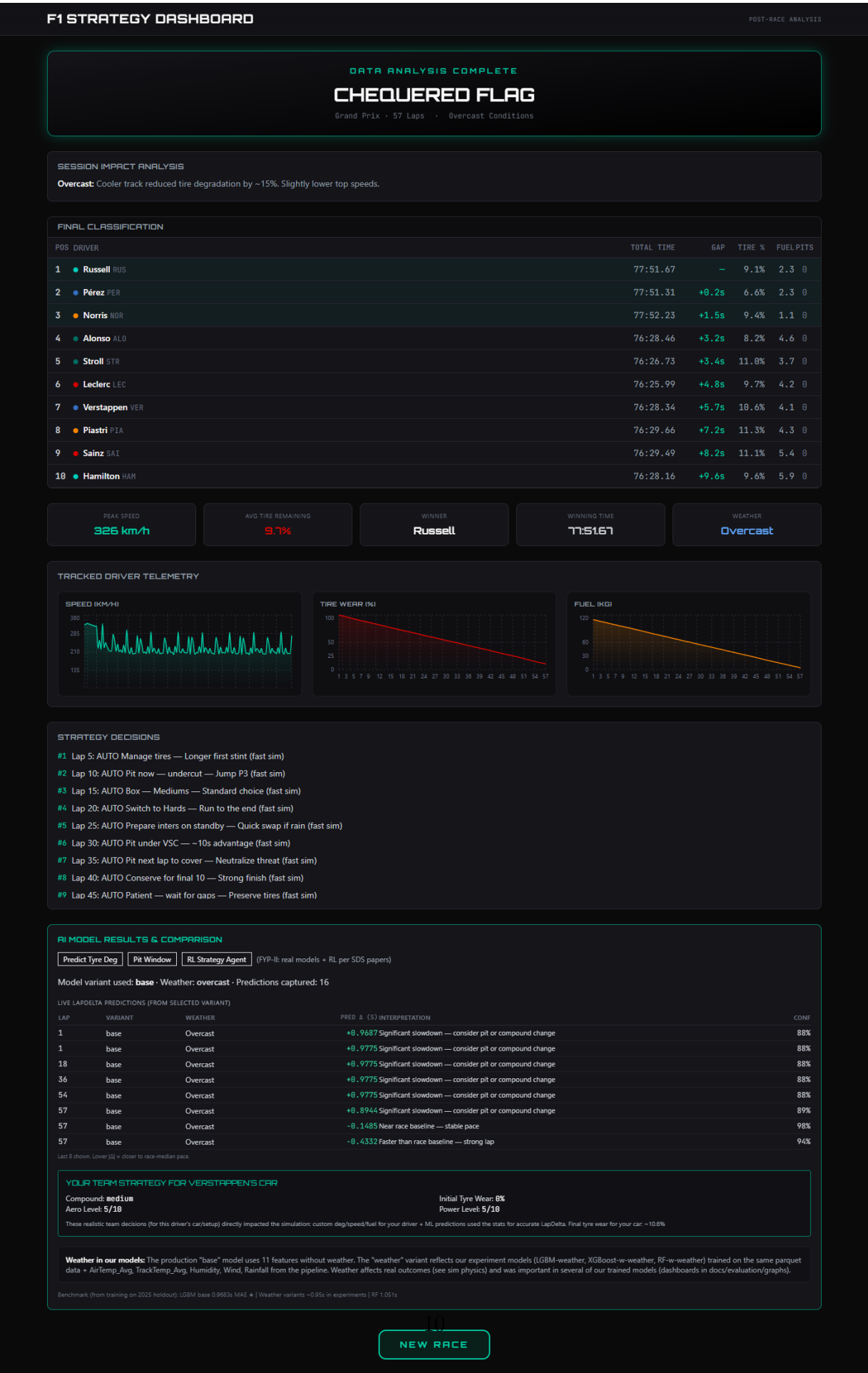
- isFastCompletingRef + state.
- On click: set state + ref, setTimeout(16, runFastBurst) so React paints the overlay *before* heavy work.
- Inside burst: for(120) tick(); if not finished setTimeout(...,10).
- Inside tick: while (lapProgress $\leq$ 100) advance lap + apply custom multipliers (only for tracked driver) + weather.
- setDrivers only every 2 laps or on finish during fast (keeps 60 fps feel for overlay text/bar).
- Decision points during fast: auto-pick highest-prob branch and continue (no pause).
- On leader lap i total: set raceFinished + clear isFast + transition in App to PostRace.

This design directly solves the “black screen / no simulating fast indicator” reported issue.

7 References & Appendices

- Original signed SRS/SDS (P80F25) for template structure and initial diagrams (retained in images/).
- Current stratbot/README.md, docs/PROJECT_CONTEXT.md, TESTING.md, model_meta.json
- All evaluation graphs live under stratbot/docs/evaluation/graphs/ (latest_* versions included here).
- 3D implementation uses three@0.167, @react-three/fiber@8, @react-three/drei@9 (legacy peer resolution via -legacy-peer-deps at install time).

8 Appendices – Additional Captures & Diagrams



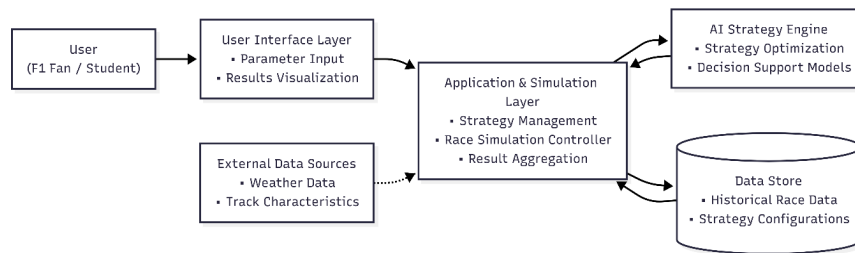


Figure 6: Reference architecture diagram from original SDS (layers still match the delivered system).



Figure 7: Reference high-level user actions from original SDS (most fully realized in current UI).

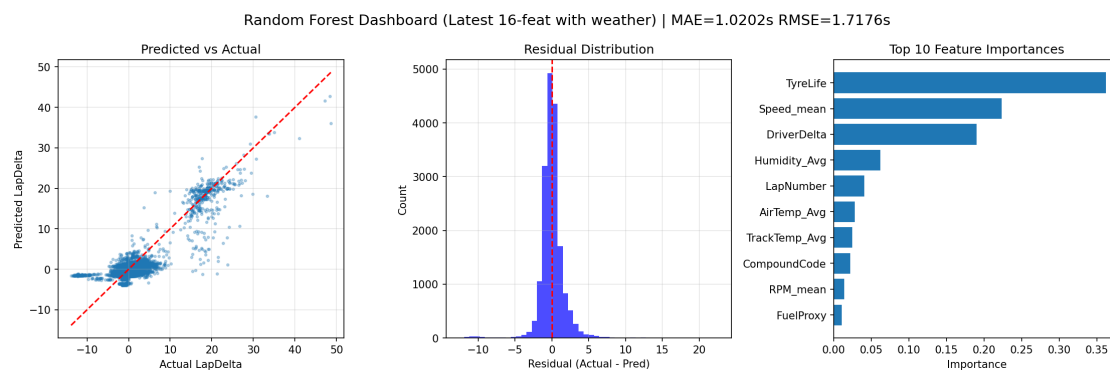


Figure 8: Production model performance dashboard (included for traceability of the numbers quoted throughout the documents).